

Q1000 SmartCity Mobility Mega Assessment

Complete PySpark Study, Implementation Flow & Code Guide

Assessment	Q1000
Domain	SmartCity Mobility / Trip Analytics
Functions	40 pure PySpark functions
Dataset	mobility_trips.csv, vehicles.csv
Primary Goal	Derived columns, filters, aggregations, joins, rankings and analytics

Each function is presented separately with its goal, implementation flow, explanation, and implementation code. The implementation code is now highlighted for quick revision.

Contents

- A) Core Trip Analytics (1–20)
- B) Inline Join + Enrichment (21–25)
- C) Unix Timestamp Conversions (26–27)
- D) Additional Analytics (28–40)
- Complete Code — `solution.py`

The assessment tests the functions independently. Where the specification requires a derived column to be created when missing, the implementation does so internally.

A) Core Trip Analytics

1. define_trip_schema()

Goal: Create the explicit schema for mobility_trips.csv.

Implementation Flow

1. Create a StructType.
2. Add all CSV columns in the same order.
3. Keep trip_date/start_ts/end_ts as strings.
4. Use DoubleType for distance, fare, discount and speed; IntegerType for harsh_brake_count.
5. Make every field nullable.

Explanation

The function only defines the schema; it does not read the CSV.

Implementation

IMPLEMENTATION CODE

```
def define_trip_schema() -> StructType:
    return StructType([
        StructField("trip_id", StringType(), True),
        StructField("user_id", StringType(), True),
        StructField("vehicle_id", StringType(), True),
        StructField("zone", StringType(), True),
        StructField("trip_date", StringType(), True),
        StructField("start_ts", StringType(), True),
        StructField("end_ts", StringType(), True),
        StructField("distance_km", DoubleType(), True),
        StructField("fare", DoubleType(), True),
        StructField("discount", DoubleType(), True),
        StructField("payment_method", StringType(), True),
        StructField("vehicle_type", StringType(), True),
        StructField("speed_kmh", DoubleType(), True),
        StructField("harsh_brake_count", IntegerType(), True),
        StructField("status", StringType(), True)
    ])

# 2. Load trips
```

2. load_trips(spark, path, schema)

Goal: Load the mobility trips CSV using the supplied schema.

Implementation Flow

1. Use spark.read.csv().
2. Set header=True.
3. Pass the supplied schema.
4. Return the DataFrame.

Explanation

The path is supplied by the caller, so the function should not hard-code the dataset path.

Implementation

IMPLEMENTATION CODE

```
def load_trips(spark: SparkSession, path: str, schema: StructType) -> DataFrame:
    return spark.read.csv(path, header=True, schema=schema)

# 3. Parse trip date
```

3. parse_trip_date(df)

Goal: Convert trip_date from string to DateType.

Implementation Flow

1. Use to_date() with yyyy-MM-dd.
2. Overwrite the existing trip_date column.
3. Return the updated DataFrame.

Explanation

This makes later date filtering and date formatting operations easier.

Implementation

IMPLEMENTATION CODE

```
def parse_trip_date(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "trip_date",
        to_date(col("trip_date"), "yyyy-MM-dd")
    )

# 4. Add trip duration in minutes
```

4. add_trip_duration_min(df)

Goal: Add integer trip duration in minutes.

Implementation Flow

1. Convert start_ts and end_ts to epoch seconds.
2. Subtract start from end.
3. Divide by 60.
4. Round to the nearest minute.
5. Cast the result to int and name it trip_duration_min.

Explanation

Duration is calculated from end time minus start time.

Implementation

IMPLEMENTATION CODE

```
def add_trip_duration_min(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "trip_duration_min",
        round(
            (unix_timestamp("end_ts") - unix_timestamp("start_ts")) / 60
        ).cast("int")
    )

# 5. Add cost per km
```

5. add_cost_per_km(df)

Goal: Add cost_per_km = fare / distance_km.

Implementation Flow

1. Create a new column with fare divided by distance_km.

2. Cast it to double.
3. Return the DataFrame.

Explanation

The assessment notes that tests normally avoid `distance_km = 0`.

Implementation

IMPLEMENTATION CODE

```
def add_cost_per_km(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "cost_per_km",
        (col("fare") / col("distance_km")).cast("double")
    )

# 6. Filter peak-hour trips
```

6. filter_peak_hour_trips(df)

Goal: Keep trips starting in peak hours.

Implementation Flow

1. Extract the hour from `start_ts`.
2. Keep hours 07–10 inclusive.
3. Also keep hours 17–20 inclusive.
4. Return only matching rows.

Explanation

The condition is: `(07 <= hour <= 10) OR (17 <= hour <= 20)`.

Implementation

IMPLEMENTATION CODE

```
def filter_peak_hour_trips(df: DataFrame) -> DataFrame:
    return df.filter(
        (
            (hour("start_ts") >= 7) &
            (hour("start_ts") <= 10)
        ) |
        (
            (hour("start_ts") >= 17) &
            (hour("start_ts") <= 20)
        )
    )

# 7. Top N users by distance
```

7. top_n_users_by_distance(df, n)

Goal: Find the top n users by total distance.

Implementation Flow

1. Group by `user_id`.
2. Sum `distance_km` as `total_distance_km`.
3. Sort `total_distance_km` descending.
4. Use `user_id` ascending as tie-breaker.
5. Limit to n.

Explanation

Both the ranking and tie-breaker must be present.

Implementation

IMPLEMENTATION CODE

```
def top_n_users_by_distance(df: DataFrame, n: int) -> DataFrame:
    return (
        df.groupBy("user_id")
            .agg(sum("distance_km").alias("total_distance_km"))
            .orderBy(desc("total_distance_km"), asc("user_id"))
            .limit(n)
    )

# 8. Average speed by zone
```

8. avg_speed_by_zone(df)

Goal: Calculate average speed for every zone.

Implementation Flow

1. Group by zone.
2. Average speed_kmh.
3. Alias it avg_speed_kmh.
4. Return the DataFrame.

Explanation

No additional ordering is specified by the assessment.

Implementation

IMPLEMENTATION CODE

```
def avg_speed_by_zone(df: DataFrame) -> DataFrame:
    return (
        df.groupBy("zone")
            .agg(avg("speed_kmh").alias("avg_speed_kmh"))
    )

# 9. Most common vehicle type
```

9. most_common_vehicle_type(df)

Goal: Find the most frequent vehicle type.

Implementation Flow

1. Group by vehicle_type.
2. Count rows.
3. Sort count descending.
4. Sort vehicle_type ascending for ties.
5. Take the first row.
6. Return empty string if there are no rows.

Explanation

The output is a Python string, not a DataFrame.

Implementation

IMPLEMENTATION CODE

```
def most_common_vehicle_type(df: DataFrame) -> str:
    row = (
        df.groupBy("vehicle_type")
        .agg(count("*").alias("count"))
        .orderBy(desc("count"), asc("vehicle_type"))
        .first()
    )

    if row is None:
        return ""

    return str(row["vehicle_type"])

# 10. Cancellation rate by zone
```

10. cancellation_rate_by_zone(df)

Goal: Calculate cancelled trips divided by total trips for each zone.

Implementation Flow

1. Group by zone.
2. Count all trips.
3. Count cancelled trips using a conditional expression.
4. Divide cancelled by total.
5. Select only zone and cancellation_rate.

Explanation

The result is a decimal rate, not multiplied by 100.

Implementation

IMPLEMENTATION CODE

```
def cancellation_rate_by_zone(df: DataFrame) -> DataFrame:
    return (
        df.groupBy("zone")
        .agg(
            count("*").alias("total"),
            sum(
                when(col("status") == "Cancelled", 1).otherwise(0)
            ).alias("cancelled")
        )
        .withColumn(
            "cancellation_rate",
            (col("cancelled") / col("total")).cast("double")
        )
        .select("zone", "cancellation_rate")
    )

# 11. High-duration trips
```

11. high_duration_trips(df, threshold_min)

Goal: Return trips whose duration is strictly greater than the threshold.

Implementation Flow

1. Check whether trip_duration_min exists.
2. If missing, calculate it.
3. Filter trip_duration_min > threshold_min.
4. Return the filtered DataFrame.

Explanation

The comparison is strictly greater than, not greater than or equal to.

Implementation

IMPLEMENTATION CODE

```
def high_duration_trips(df: DataFrame, threshold_min: int) -> DataFrame:
    if "trip_duration_min" not in df.columns:
        df = add_trip_duration_min(df)

    return df.filter(col("trip_duration_min") > threshold_min)

# 12. Count active vehicles by zone
```

12. count_active_vehicles_by_zone(vehicles_df)

Goal: Count active vehicles by their home zone.

Implementation Flow

1. Filter status == Active.
2. Group by home_zone.
3. Count distinct vehicle_id.
4. Alias as active_vehicles.
5. Rename home_zone to zone.

Explanation

Only active vehicles are included.

Implementation

IMPLEMENTATION CODE

```
def count_active_vehicles_by_zone(vehicles_df: DataFrame) -> DataFrame:
    return (
        vehicles_df
        .filter(col("status") == "Active")
        .groupBy("home_zone")
        .agg(countDistinct("vehicle_id").alias("active_vehicles"))
        .withColumnRenamed("home_zone", "zone")
    )

# 13. Daily net revenue trend
```

13. daily_net_revenue_trend(df)

Goal: Calculate daily net revenue.

Implementation Flow

1. Create net_revenue = fare - discount.
2. Group by trip_date.
3. Sum net_revenue as daily_net_revenue.
4. Order by trip_date ascending.

Explanation

The calculation is performed per row before daily aggregation.

Implementation

IMPLEMENTATION CODE

```
def daily_net_revenue_trend(df: DataFrame) -> DataFrame:
    return (
        df.withColumn(
            "net_revenue",
            (col("fare") - col("discount")).cast("double")
        )
        .groupBy("trip_date")
        .agg(sum("net_revenue").alias("daily_net_revenue"))
        .orderBy(asc("trip_date"))
    )

# 14. Top vehicle by net revenue
```

14. top_vehicle_by_net_revenue(df)

Goal: Find the vehicle with the highest total net revenue.

Implementation Flow

1. Calculate fare - discount.
2. Group by vehicle_id.
3. Sum as total_net_revenue.
4. Sort revenue descending.
5. Tie-break vehicle_id ascending.
6. Take the first row.
7. Return (vehicle_id, revenue), or ("", 0.0) if empty.

Explanation

The result is a Python tuple with string and float.

Implementation

IMPLEMENTATION CODE

```
def top_vehicle_by_net_revenue(df: DataFrame) -> Tuple[str, float]:
    row = (
        df.withColumn(
            "net_revenue",
            col("fare") - col("discount")
        )
        .groupBy("vehicle_id")
        .agg(sum("net_revenue").alias("total_net_revenue"))
        .orderBy(desc("total_net_revenue"), asc("vehicle_id"))
        .first()
    )

    if row is None:
        return ("", 0.0)

    return (
        str(row["vehicle_id"]),
        float(row["total_net_revenue"])
    )

# 15. List zones
```

15. list_zones(df)

Goal: Return unique zones in ascending order.

Implementation Flow

1. Select zone.
2. Remove duplicates.

3. Collect to the driver.
4. Convert values to strings.
5. Sort in Python.
6. Return the list.

Explanation

The assessment explicitly asks for a Python List[str].

Implementation

IMPLEMENTATION CODE

```
def list_zones(df: DataFrame) -> List[str]:
    rows = df.select("zone").distinct().collect()
    zones = [str(row["zone"]) for row in rows]
    return sorted(zones)
```

16. Trips in date range

16. trips_in_date_range(df, start, end)

Goal: Return trips whose trip_date is between start and end, inclusive.

Implementation Flow

1. Ensure trip_date is DateType.
2. Convert start and end to date literals.
3. Use between().
4. Return the filtered DataFrame.

Explanation

Both endpoints are included.

Implementation

IMPLEMENTATION CODE

```
def trips_in_date_range(df: DataFrame, start: str, end: str) -> DataFrame:
    if dict(df.dtypes)["trip_date"] != "date":
        df = df.withColumn(
            "trip_date",
            to_date(col("trip_date"), "yyyy-MM-dd")
        )

    start_date = to_date(lit(start), "yyyy-MM-dd")
    end_date = to_date(lit(end), "yyyy-MM-dd")

    return df.filter(col("trip_date").between(start_date, end_date))
```

17. Flag safety risk

17. flag_safety_risk(df, speed_threshold, brake_threshold)

Goal: Add a boolean safety-risk flag.

Implementation Flow

1. Check speed_kmh > speed_threshold.
2. Check harsh_brake_count > brake_threshold.
3. Combine with OR.
4. Store the boolean in is_risky.

Explanation

Either condition being true makes the trip risky.

Implementation

IMPLEMENTATION CODE

```
def flag_safety_risk(
    df: DataFrame,
    speed_threshold: float,
    brake_threshold: int
) -> DataFrame:
    return df.withColumn(
        "is_risky",
        (
            (col("speed_kmh") > speed_threshold) |
            (col("harsh_brake_count") > brake_threshold)
        )
    )

# 18. Top N risky vehicles
```

18. top_n_risky_vehicles(df, n)

Goal: Find vehicles with the most risky trips.

Implementation Flow

1. Filter is_risky == True.
2. Group by vehicle_id.
3. Count risky rows.
4. Sort risky_count descending.
5. Tie-break vehicle_id ascending.
6. Limit to n.

Explanation

The function expects is_risky to already be available.

Implementation

IMPLEMENTATION CODE

```
def top_n_risky_vehicles(df: DataFrame, n: int) -> DataFrame:
    return (
        df.filter(col("is_risky") == True)
        .groupBy("vehicle_id")
        .agg(count("*").alias("risky_count"))
        .orderBy(desc("risky_count"), asc("vehicle_id"))
        .limit(n)
    )

# 19. Average fare by payment method
```

19. avg_fare_by_payment_method(df)

Goal: Calculate average fare for each payment method.

Implementation Flow

1. Group by payment_method.
2. Average fare.
3. Alias as avg_fare.
4. Return the DataFrame.

Explanation

No ordering is specified.

Implementation

IMPLEMENTATION CODE

```
def avg_fare_by_payment_method(df: DataFrame) -> DataFrame:
    return (
        df.groupBy("payment_method")
            .agg(avg("fare").alias("avg_fare"))
    )

# 20. Get longest trip
```

20. get_longest_trip(df)

Goal: Return the trip with the greatest duration.

Implementation Flow

1. Ensure trip_duration_min exists; calculate it if missing.
2. Sort duration descending.
3. Tie-break trip_id ascending.
4. Select trip_id and duration.
5. Return ("", 0) if empty.

Explanation

The result is a Python tuple (trip_id, duration).

Implementation

IMPLEMENTATION CODE

```
def get_longest_trip(df: DataFrame) -> Tuple[str, int]:
    if "trip_duration_min" not in df.columns:
        df = add_trip_duration_min(df)

    row = (
        df.orderBy(desc("trip_duration_min"), asc("trip_id"))
            .select("trip_id", "trip_duration_min")
            .first()
    )

    if row is None:
        return ("", 0)

    return (
        str(row["trip_id"]),
        int(row["trip_duration_min"])
    )

# =====
# B) Inline Join + Enrichment
# =====

# 21. Define user profile schema
```

B) Inline Join + Enrichment

21. define_user_profile_schema()

Goal: Create the schema for inline user profiles.

Implementation Flow

1. Create StructType.
2. Add user_id, first_name, last_name, full_name and plan_id.
3. Use StringType and nullable=True.

Explanation

All five profile fields are strings.

Implementation

IMPLEMENTATION CODE

```
def define_user_profile_schema() -> StructType:
    return StructType([
        StructField("user_id", StringType(), True),
        StructField("first_name", StringType(), True),
        StructField("last_name", StringType(), True),
        StructField("full_name", StringType(), True),
        StructField("plan_id", StringType(), True)
    ])

# 22. Define plan schema
# Note: The assessment names this function define_vehicle_schema().
```

22. define_vehicle_schema()

Goal: Create the schema specified for the inline plans lookup.

Implementation Flow

1. Create StructType.
2. Add plan_id and plan_name.
3. Use StringType and nullable=True.

Explanation

Despite its function name, the assessment specifies plan_id and plan_name here.

Implementation

IMPLEMENTATION CODE

```
def define_vehicle_schema() -> StructType:
    return StructType([
        StructField("plan_id", StringType(), True),
        StructField("plan_name", StringType(), True)
    ])

# 23. Load inline profiles and plans
```

23. load_inline_profiles_and_plans(...)

Goal: Create two small lookup DataFrames.

Implementation Flow

1. Create the four profile tuples given in the assessment.
2. Create the two plan tuples.

3. Use `spark.createDataFrame()` with the supplied schemas.

4. Return `(profiles_df, plans_df)`.

Explanation

The inline rows are U1–U4 and P1–P2 exactly as given in the assessment.

Implementation

IMPLEMENTATION CODE

```
def load_inline_profiles_and_plans(
    spark: SparkSession,
    profile_schema: StructType,
    plan_schema: StructType
) -> Tuple[DataFrame, DataFrame]:

    profiles_rows = [
        ("U1", "Aarav", "Sharma", None, "P2"),
        ("U2", "Diya", "Iyer", "Diya Iyer", "P1"),
        ("U3", "Kabir", "Singh", None, "P1"),
        ("U4", "Meera", "Nair", None, "P2")
    ]

    plans_rows = [
        ("P1", "Standard"),
        ("P2", "Premium")
    ]

    profiles_df = spark.createDataFrame(
        profiles_rows,
        schema=profile_schema
    )

    plans_df = spark.createDataFrame(
        plans_rows,
        schema=plan_schema
    )

    return profiles_df, plans_df

# 24. Join profiles with plans
```

24. join_profiles_with_plans(profiles_df, plans_df)

Goal: Left join profiles with plans on `plan_id`.

Implementation Flow

1. Perform a left join.
2. Join on `plan_id`.
3. Select `user_id`, `first_name`, `last_name`, `full_name`, `plan_id` and `plan_name`.
4. Return the DataFrame.

Explanation

The output columns must match the specified order.

Implementation

IMPLEMENTATION CODE

```
def join_profiles_with_plans(
    profiles_df: DataFrame,
    plans_df: DataFrame
) -> DataFrame:
    return (
        profiles_df
        .join(
            plans_df,
            profiles_df["plan_id"] == plans_df["plan_id"],
            "left"
        )
        .select(
            profiles_df["user_id"],
            profiles_df["first_name"],
            profiles_df["last_name"],
            profiles_df["full_name"],
            profiles_df["plan_id"],
            plans_df["plan_name"]
        )
    )

# 25. Enrich full name
```

25. enrich_full_name(profile_joined_df)

Goal: Fill missing or blank full_name values.

Implementation Flow

1. Check full_name is null OR trimmed length is zero.
2. If missing, concatenate first_name and last_name with a space.
3. Otherwise preserve the existing full_name.
4. Return the DataFrame.

Explanation

Existing nonblank full names are not changed.

Implementation

IMPLEMENTATION CODE

```
def enrich_full_name(profile_joined_df: DataFrame) -> DataFrame:
    return profile_joined_df.withColumn(
        "full_name",
        when(
            col("full_name").isNull() |
            (length(trim(col("full_name")))) == 0,
            concat_ws(" ", col("first_name"), col("last_name"))
        ).otherwise(col("full_name"))
    )

# =====
# C) Unix Timestamp Conversions
# =====

# 26. Add start epoch seconds
```

C) Unix Timestamp Conversions

26. add_start_epoch_seconds(df)

Goal: Add epoch seconds for start_ts.

Implementation Flow

1. Convert start_ts to Unix time.
2. Cast to long.
3. Store in start_epoch_seconds.

Explanation

The new column contains integer-like epoch seconds.

Implementation

IMPLEMENTATION CODE

```
def add_start_epoch_seconds(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "start_epoch_seconds",
        unix_timestamp("start_ts").cast("long")
    )

# 27. Add start timestamp from epoch
```

27. add_start_ts_from_epoch(df)

Goal: Convert start_epoch_seconds back to a timestamp.

Implementation Flow

1. Read start_epoch_seconds.
2. Use from_unixtime().
3. Cast to timestamp.
4. Store as start_ts_from_epoch.

Explanation

This reverses the epoch conversion at timestamp precision supported by Spark.

Implementation

IMPLEMENTATION CODE

```
def add_start_ts_from_epoch(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "start_ts_from_epoch",
        from_unixtime(col("start_epoch_seconds")).cast("timestamp")
    )

# =====
# D) Additional Analytics
# =====

# 28. Revenue share by vehicle type
```

D) Additional Analytics

28. revenue_share_by_vehicle_type(df)

Goal: Return vehicle types ordered by their share of total revenue.

Implementation Flow

1. Group by vehicle_type.
2. Sum rev as total_rev.
3. Compute the grand total.
4. If grand total is null or zero, return [].
5. Compute share = total_rev / grand total.
6. Sort share descending and vehicle_type ascending.
7. Collect vehicle_type into a Python list.

Explanation

The input column for revenue is named rev.

Implementation

IMPLEMENTATION CODE

```
def revenue_share_by_vehicle_type(df: DataFrame) -> List[str]:
    revenue = (
        df.groupBy("vehicle_type")
        .agg(sum("rev").alias("total_rev"))
    )

    total = (
        revenue.agg(sum("total_rev").alias("grand_total"))
        .first()["grand_total"]
    )

    if total is None or total == 0:
        return []

    result = (
        revenue
        .withColumn("share", col("total_rev") / lit(total))
        .orderBy(desc("share"), asc("vehicle_type"))
        .select("vehicle_type")
        .collect()
    )

    return [str(row["vehicle_type"]) for row in result]

# 29. Busiest zone by trips
```

29. busiest_zone_by_trips(df)

Goal: Find the zone with the maximum number of trips.

Implementation Flow

1. Group by zone.
2. Count rows.
3. Sort count descending.
4. Tie-break zone ascending.
5. Take first row.
6. Return ("", 0) if empty.

Explanation

The result is a Python tuple.

Implementation

IMPLEMENTATION CODE

```
def busiest_zone_by_trips(df: DataFrame) -> Tuple[str, int]:
    row = (
        df.groupBy("zone")
        .agg(count("*").alias("count"))
        .orderBy(desc("count"), asc("zone"))
        .first()
    )

    if row is None:
        return ("", 0)

    return (
        str(row["zone"]),
        int(row["count"])
    )

# 30. Completed trips by zone
```

30. completed_trips_by_zone(df)

Goal: Count completed trips per zone.

Implementation Flow

1. Filter status == Completed.
2. Group by zone.
3. Count as completed_trips.
4. Sort completed_trips descending.
5. Tie-break zone ascending.

Explanation

Only completed trips contribute to the count.

Implementation

IMPLEMENTATION CODE

```
def completed_trips_by_zone(df: DataFrame) -> DataFrame:
    return (
        df.filter(col("status") == "Completed")
        .groupBy("zone")
        .agg(count("*").alias("completed_trips"))
        .orderBy(desc("completed_trips"), asc("zone"))
    )

# 31. Add utilization score
```

31. add_utilization_score(df)

Goal: Add utilization_score = distance_km * trip_duration_min.

Implementation Flow

1. Ensure trip_duration_min exists.
2. If missing, calculate it.
3. Multiply distance_km by trip_duration_min.
4. Cast to double.

5. Return the DataFrame.

Explanation

The assessment defines utilization as a product of distance and duration.

Implementation

IMPLEMENTATION CODE

```
def add_utilization_score(df: DataFrame) -> DataFrame:
    if "trip_duration_min" not in df.columns:
        df = add_trip_duration_min(df)

    return df.withColumn(
        "utilization_score",
        (col("distance_km") * col("trip_duration_min")).cast("double")
    )

# 32. Top N users by utilization
```

32. top_n_users_by_utilization(df, n)

Goal: Find users with the highest total utilization.

Implementation Flow

1. Ensure utilization_score exists.
2. Group by user_id.
3. Sum utilization_score.
4. Alias total_utilization.
5. Sort descending.
6. Tie-break user_id ascending.
7. Limit n.

Explanation

This is an aggregation over user-level utilization.

Implementation

IMPLEMENTATION CODE

```
def top_n_users_by_utilization(
    df: DataFrame,
    n: int
) -> DataFrame:
    if "utilization_score" not in df.columns:
        df = add_utilization_score(df)

    return (
        df.groupBy("user_id")
        .agg(sum("utilization_score").alias("total_utilization"))
        .orderBy(desc("total_utilization"), asc("user_id"))
        .limit(n)
    )

# 33. Median duration by zone
```

33. median_duration_by_zone(df)

Goal: Calculate approximate median duration for each zone.

Implementation Flow

1. Ensure trip_duration_min exists.

2. Group by zone.
3. Use percentile_approx at 0.5.
4. Alias median_duration_min.
5. Return the DataFrame.

Explanation

The assessment specifically requests approximate median aggregation.

Implementation

IMPLEMENTATION CODE

```
def median_duration_by_zone(df: DataFrame) -> DataFrame:
    if "trip_duration_min" not in df.columns:
        df = add_trip_duration_min(df)

    return (
        df.groupBy("zone")
        .agg(
            percentile_approx(
                "trip_duration_min",
                0.5
            ).alias("median_duration_min")
        )
    )

# 34. Min-max normalize fare
```

34. minmax_normalize_fare(df)

Goal: Normalize fare to the range implied by min-max normalization.

Implementation Flow

1. Aggregate global minimum and maximum fare once.
2. If either is missing or min == max, set fare_norm to 0.0.
3. Otherwise calculate (fare - min) / (max - min).
4. Return the DataFrame.

Explanation

The global min/max are collected once to the driver.

Implementation

IMPLEMENTATION CODE

```
def minmax_normalize_fare(df: DataFrame) -> DataFrame:
    result = df.agg(
        min("fare").alias("min_fare"),
        max("fare").alias("max_fare")
    ).first()

    min_fare = result["min_fare"]
    max_fare = result["max_fare"]

    if (
        min_fare is None or
        max_fare is None or
        max_fare == min_fare
    ):
        return df.withColumn("fare_norm", lit(0.0))

    return df.withColumn(
        "fare_norm",
        (
            (col("fare") - lit(min_fare)) /
            lit(max_fare - min_fare)
        ).cast("double")
    )

# 35. Detect outlier fares
```

35. detect_outlier_fares(df)

Goal: Flag fares above mean + 2 * population standard deviation.

Implementation Flow

1. Aggregate global mean and stddev_pop.
2. If either is missing, set is_outlier to False.
3. Otherwise calculate the threshold.
4. Set is_outlier to fare > threshold.

Explanation

The threshold is based on the population standard deviation, as specified.

Implementation

IMPLEMENTATION CODE

```
def detect_outlier_fares(df: DataFrame) -> DataFrame:
    result = df.agg(
        avg("fare").alias("mean_fare"),
        stddev_pop("fare").alias("std_fare")
    ).first()

    mean_fare = result["mean_fare"]
    std_fare = result["std_fare"]

    if mean_fare is None or std_fare is None:
        return df.withColumn("is_outlier", lit(False))

    threshold = mean_fare + (2 * std_fare)

    return df.withColumn(
        "is_outlier",
        col("fare") > lit(threshold)
    )

# 36. Monthly net revenue by zone
```

36. monthly_net_revenue_by_zone(df)

Goal: Calculate monthly net revenue for each zone.

Implementation Flow

1. Ensure trip_date is DateType if needed.
2. Calculate fare - discount.
3. Format trip_date as yyyy-MM.
4. Group by month and zone.
5. Sum as monthly_net_revenue.
6. Order month ascending then zone ascending.

Explanation

The output month is a string formatted yyyy-MM.

Implementation

IMPLEMENTATION CODE

```
def monthly_net_revenue_by_zone(df: DataFrame) -> DataFrame:
    if dict(df.dtypes)["trip_date"] != "date":
        df = df.withColumn(
            "trip_date",
            to_date(col("trip_date"), "yyyy-MM-dd")
        )

    return (
        df.withColumn(
            "net_revenue",
            col("fare") - col("discount")
        )
        .withColumn(
            "month",
            date_format(col("trip_date"), "yyyy-MM")
        )
        .groupBy("month", "zone")
        .agg(
            sum("net_revenue").alias("monthly_net_revenue")
        )
        .orderBy(asc("month"), asc("zone"))
    )

# 37. Weekday peak trip counts
```

37. weekday_peak_trip_counts(df)

Goal: Count peak-hour trips by weekday label.

Implementation Flow

1. Reuse peak-hour filtering.
2. Ensure trip_date is DateType.
3. Extract weekday label such as Mon/Tue.
4. Group by weekday.
5. Count as peak_trips.

Explanation

The assessment does not specify a weekday ordering.

Implementation

IMPLEMENTATION CODE

```
def weekday_peak_trip_counts(df: DataFrame) -> DataFrame:
    peak_df = filter_peak_hour_trips(df)

    if dict(peak_df.dtypes)["trip_date"] != "date":
        peak_df = peak_df.withColumn(
            "trip_date",
            to_date(col("trip_date"), "yyyy-MM-dd")
        )

    return (
        peak_df
        .withColumn(
            "weekday",
            date_format(col("trip_date"), "E")
        )
        .groupBy("weekday")
        .agg(count("*").alias("peak_trips"))
    )

# 38. Pivot payment counts by zone
```

38. pivot_payment_counts_by_zone(df)

Goal: Create a zone-by-payment-method count pivot.

Implementation Flow

1. Group by zone.
2. Pivot on payment_method.
3. Count rows.
4. Fill missing values with 0.
5. Return the pivoted DataFrame.

Explanation

The payment methods become pivot columns.

Implementation

IMPLEMENTATION CODE

```
def pivot_payment_counts_by_zone(df: DataFrame) -> DataFrame:
    return (
        df.groupBy("zone")
        .pivot("payment_method")
        .count()
        .fillna(0)
    )

# 39. Calculate net revenue
```

39. calculate_net_revenue(df)

Goal: Add or overwrite net_revenue.

Implementation Flow

1. Subtract discount from fare.
2. Cast the result to double.
3. Store it in net_revenue.
4. Return the DataFrame.

Explanation

This is a row-level derived column.

Implementation

IMPLEMENTATION CODE

```
def calculate_net_revenue(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "net_revenue",
        (col("fare") - col("discount")).cast("double")
    )

# 40. Flag service due
```

40. flag_service_due(df, due_days)

Goal: Flag vehicles whose service is overdue.

Implementation Flow

1. Ensure last_service_date is DateType if needed.
2. Calculate datediff(current_date(), last_service_date).
3. Set is_service_due when the difference is greater than due_days.
4. Return the DataFrame.

Explanation

The comparison is strictly greater than due_days.

Implementation

IMPLEMENTATION CODE

```
def flag_service_due(
    df: DataFrame,
    due_days: int
) -> DataFrame:
    if dict(df.dtypes)["last_service_date"] != "date":
        df = df.withColumn(
            "last_service_date",
            to_date(col("last_service_date"), "yyyy-MM-dd")
        )

    days = datediff(
        current_date(),
        col("last_service_date")
    )

    return df.withColumn(
        "is_service_due",
        days > due_days
    )
```

Complete Code — solution.py

Complete 40-function implementation. The code blocks are highlighted for easier study and copying.

IMPLEMENTATION CODE

```
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql.functions import *
from pyspark.sql.types import *
from typing import List, Tuple

# =====
# A) Core Trip Analytics
# =====

# 1. Define trip schema
def define_trip_schema() -> StructType:
    return StructType([
        StructField("trip_id", StringType(), True),
        StructField("user_id", StringType(), True),
        StructField("vehicle_id", StringType(), True),
        StructField("zone", StringType(), True),
        StructField("trip_date", StringType(), True),
        StructField("start_ts", StringType(), True),
        StructField("end_ts", StringType(), True),
        StructField("distance_km", DoubleType(), True),
        StructField("fare", DoubleType(), True),
        StructField("discount", DoubleType(), True),
        StructField("payment_method", StringType(), True),
        StructField("vehicle_type", StringType(), True),
        StructField("speed_kmh", DoubleType(), True),
        StructField("harsh_brake_count", IntegerType(), True),
        StructField("status", StringType(), True)
    ])

# 2. Load trips
def load_trips(spark: SparkSession, path: str, schema: StructType) -> DataFrame:
    return spark.read.csv(path, header=True, schema=schema)

# 3. Parse trip date
def parse_trip_date(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "trip_date",
        to_date(col("trip_date"), "yyyy-MM-dd")
    )

# 4. Add trip duration in minutes
def add_trip_duration_min(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "trip_duration_min",
        round(
            (unix_timestamp("end_ts") - unix_timestamp("start_ts")) / 60
        ).cast("int")
    )

# 5. Add cost per km
```

IMPLEMENTATION CODE

```
def add_cost_per_km(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "cost_per_km",
        (col("fare") / col("distance_km")).cast("double")
    )

# 6. Filter peak-hour trips
def filter_peak_hour_trips(df: DataFrame) -> DataFrame:
    return df.filter(
        (
            (hour("start_ts") >= 7) &
            (hour("start_ts") <= 10)
        ) |
        (
            (hour("start_ts") >= 17) &
            (hour("start_ts") <= 20)
        )
    )

# 7. Top N users by distance
def top_n_users_by_distance(df: DataFrame, n: int) -> DataFrame:
    return (
        df.groupBy("user_id")
        .agg(sum("distance_km").alias("total_distance_km"))
        .orderBy(desc("total_distance_km"), asc("user_id"))
        .limit(n)
    )

# 8. Average speed by zone
def avg_speed_by_zone(df: DataFrame) -> DataFrame:
    return (
        df.groupBy("zone")
        .agg(avg("speed_kmh").alias("avg_speed_kmh"))
    )

# 9. Most common vehicle type
def most_common_vehicle_type(df: DataFrame) -> str:
    row = (
        df.groupBy("vehicle_type")
        .agg(count("*").alias("count"))
        .orderBy(desc("count"), asc("vehicle_type"))
        .first()
    )

    if row is None:
        return ""

    return str(row["vehicle_type"])

# 10. Cancellation rate by zone
```

IMPLEMENTATION CODE

```
def cancellation_rate_by_zone(df: DataFrame) -> DataFrame:
    return (
        df.groupBy("zone")
        .agg(
            count("*").alias("total"),
            sum(
                when(col("status") == "Cancelled", 1).otherwise(0)
            ).alias("cancelled")
        )
        .withColumn(
            "cancellation_rate",
            (col("cancelled") / col("total")).cast("double")
        )
        .select("zone", "cancellation_rate")
    )

# 11. High-duration trips
def high_duration_trips(df: DataFrame, threshold_min: int) -> DataFrame:
    if "trip_duration_min" not in df.columns:
        df = add_trip_duration_min(df)

    return df.filter(col("trip_duration_min") > threshold_min)

# 12. Count active vehicles by zone
def count_active_vehicles_by_zone(vehicles_df: DataFrame) -> DataFrame:
    return (
        vehicles_df
        .filter(col("status") == "Active")
        .groupBy("home_zone")
        .agg(countDistinct("vehicle_id").alias("active_vehicles"))
        .withColumnRenamed("home_zone", "zone")
    )

# 13. Daily net revenue trend
def daily_net_revenue_trend(df: DataFrame) -> DataFrame:
    return (
        df.withColumn(
            "net_revenue",
            (col("fare") - col("discount")).cast("double")
        )
        .groupBy("trip_date")
        .agg(sum("net_revenue").alias("daily_net_revenue"))
        .orderBy(asc("trip_date"))
    )

# 14. Top vehicle by net revenue
def top_vehicle_by_net_revenue(df: DataFrame) -> Tuple[str, float]:
    row = (
        df.withColumn(
            "net_revenue",
            col("fare") - col("discount")
        )
    )
```

IMPLEMENTATION CODE

```
)
.groupBy("vehicle_id")
.agg(sum("net_revenue").alias("total_net_revenue"))
.orderBy(desc("total_net_revenue"), asc("vehicle_id"))
.first()
)

if row is None:
    return ("", 0.0)

return (
    str(row["vehicle_id"]),
    float(row["total_net_revenue"])
)

# 15. List zones
def list_zones(df: DataFrame) -> List[str]:
    rows = df.select("zone").distinct().collect()
    zones = [str(row["zone"]) for row in rows]
    return sorted(zones)

# 16. Trips in date range
def trips_in_date_range(df: DataFrame, start: str, end: str) -> DataFrame:
    if dict(df.dtypes)["trip_date"] != "date":
        df = df.withColumn(
            "trip_date",
            to_date(col("trip_date"), "yyyy-MM-dd")
        )

    start_date = to_date(lit(start), "yyyy-MM-dd")
    end_date = to_date(lit(end), "yyyy-MM-dd")

    return df.filter(col("trip_date").between(start_date, end_date))

# 17. Flag safety risk
def flag_safety_risk(
    df: DataFrame,
    speed_threshold: float,
    brake_threshold: int
) -> DataFrame:
    return df.withColumn(
        "is_risky",
        (
            (col("speed_kmh") > speed_threshold) |
            (col("harsh_brake_count") > brake_threshold)
        )
    )

# 18. Top N risky vehicles
def top_n_risky_vehicles(df: DataFrame, n: int) -> DataFrame:
    return (
```

IMPLEMENTATION CODE

```
        df.filter(col("is_risky") == True)
        .groupBy("vehicle_id")
        .agg(count("").alias("risky_count"))
        .orderBy(desc("risky_count"), asc("vehicle_id"))
        .limit(n)
    )

# 19. Average fare by payment method
def avg_fare_by_payment_method(df: DataFrame) -> DataFrame:
    return (
        df.groupBy("payment_method")
        .agg(avg("fare").alias("avg_fare"))
    )

# 20. Get longest trip
def get_longest_trip(df: DataFrame) -> Tuple[str, int]:
    if "trip_duration_min" not in df.columns:
        df = add_trip_duration_min(df)

    row = (
        df.orderBy(desc("trip_duration_min"), asc("trip_id"))
        .select("trip_id", "trip_duration_min")
        .first()
    )

    if row is None:
        return ("", 0)

    return (
        str(row["trip_id"]),
        int(row["trip_duration_min"])
    )

# =====
# B) Inline Join + Enrichment
# =====

# 21. Define user profile schema
def define_user_profile_schema() -> StructType:
    return StructType([
        StructField("user_id", StringType(), True),
        StructField("first_name", StringType(), True),
        StructField("last_name", StringType(), True),
        StructField("full_name", StringType(), True),
        StructField("plan_id", StringType(), True)
    ])

# 22. Define plan schema
# Note: The assessment names this function define_vehicle_schema().
def define_vehicle_schema() -> StructType:
    return StructType([
```

IMPLEMENTATION CODE

```
        StructField("plan_id", StringType(), True),
        StructField("plan_name", StringType(), True)
    ])

# 23. Load inline profiles and plans
def load_inline_profiles_and_plans(
    spark: SparkSession,
    profile_schema: StructType,
    plan_schema: StructType
) -> Tuple[DataFrame, DataFrame]:

    profiles_rows = [
        ("U1", "Aarav", "Sharma", None, "P2"),
        ("U2", "Diya", "Iyer", "Diya Iyer", "P1"),
        ("U3", "Kabir", "Singh", None, "P1"),
        ("U4", "Meera", "Nair", None, "P2")
    ]

    plans_rows = [
        ("P1", "Standard"),
        ("P2", "Premium")
    ]

    profiles_df = spark.createDataFrame(
        profiles_rows,
        schema=profile_schema
    )

    plans_df = spark.createDataFrame(
        plans_rows,
        schema=plan_schema
    )

    return profiles_df, plans_df

# 24. Join profiles with plans
def join_profiles_with_plans(
    profiles_df: DataFrame,
    plans_df: DataFrame
) -> DataFrame:
    return (
        profiles_df
        .join(
            plans_df,
            profiles_df["plan_id"] == plans_df["plan_id"],
            "left"
        )
        .select(
            profiles_df["user_id"],
            profiles_df["first_name"],
            profiles_df["last_name"],
            profiles_df["full_name"],
            profiles_df["plan_id"],
        )
    )
```

IMPLEMENTATION CODE

```
        plans_df["plan_name"]
    )
)

# 25. Enrich full name
def enrich_full_name(profile_joined_df: DataFrame) -> DataFrame:
    return profile_joined_df.withColumn(
        "full_name",
        when(
            col("full_name").isNull() |
            (length(trim(col("full_name")))) == 0),
            concat_ws(" ", col("first_name"), col("last_name"))
        ).otherwise(col("full_name"))
    )

# =====
# C) Unix Timestamp Conversions
# =====

# 26. Add start epoch seconds
def add_start_epoch_seconds(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "start_epoch_seconds",
        unix_timestamp("start_ts").cast("long")
    )

# 27. Add start timestamp from epoch
def add_start_ts_from_epoch(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "start_ts_from_epoch",
        from_unixtime(col("start_epoch_seconds")).cast("timestamp")
    )

# =====
# D) Additional Analytics
# =====

# 28. Revenue share by vehicle type
def revenue_share_by_vehicle_type(df: DataFrame) -> List[str]:
    revenue = (
        df.groupBy("vehicle_type")
        .agg(sum("rev").alias("total_rev"))
    )

    total = (
        revenue.agg(sum("total_rev").alias("grand_total"))
        .first()["grand_total"]
    )

    if total is None or total == 0:
        return []
```

IMPLEMENTATION CODE

```
result = (
    revenue
    .withColumn("share", col("total_rev") / lit(total))
    .orderBy(desc("share"), asc("vehicle_type"))
    .select("vehicle_type")
    .collect()
)

return [str(row["vehicle_type"]) for row in result]

# 29. Busiest zone by trips
def busiest_zone_by_trips(df: DataFrame) -> Tuple[str, int]:
    row = (
        df.groupBy("zone")
        .agg(count("*").alias("count"))
        .orderBy(desc("count"), asc("zone"))
        .first()
    )

    if row is None:
        return ("", 0)

    return (
        str(row["zone"]),
        int(row["count"])
    )

# 30. Completed trips by zone
def completed_trips_by_zone(df: DataFrame) -> DataFrame:
    return (
        df.filter(col("status") == "Completed")
        .groupBy("zone")
        .agg(count("*").alias("completed_trips"))
        .orderBy(desc("completed_trips"), asc("zone"))
    )

# 31. Add utilization score
def add_utilization_score(df: DataFrame) -> DataFrame:
    if "trip_duration_min" not in df.columns:
        df = add_trip_duration_min(df)

    return df.withColumn(
        "utilization_score",
        (col("distance_km") * col("trip_duration_min")).cast("double")
    )

# 32. Top N users by utilization
def top_n_users_by_utilization(
    df: DataFrame,
    n: int

```

IMPLEMENTATION CODE

```
) -> DataFrame:
  if "utilization_score" not in df.columns:
    df = add_utilization_score(df)

  return (
    df.groupBy("user_id")
      .agg(sum("utilization_score").alias("total_utilization"))
      .orderBy(desc("total_utilization"), asc("user_id"))
      .limit(n)
  )

# 33. Median duration by zone
def median_duration_by_zone(df: DataFrame) -> DataFrame:
  if "trip_duration_min" not in df.columns:
    df = add_trip_duration_min(df)

  return (
    df.groupBy("zone")
      .agg(
        percentile_approx(
          "trip_duration_min",
          0.5
        ).alias("median_duration_min")
      )
  )

# 34. Min-max normalize fare
def minmax_normalize_fare(df: DataFrame) -> DataFrame:
  result = df.agg(
    min("fare").alias("min_fare"),
    max("fare").alias("max_fare")
  ).first()

  min_fare = result["min_fare"]
  max_fare = result["max_fare"]

  if (
    min_fare is None or
    max_fare is None or
    max_fare == min_fare
  ):
    return df.withColumn("fare_norm", lit(0.0))

  return df.withColumn(
    "fare_norm",
    (
      (col("fare") - lit(min_fare)) /
      lit(max_fare - min_fare)
    ).cast("double")
  )

# 35. Detect outlier fares
```

IMPLEMENTATION CODE

```
def detect_outlier_fares(df: DataFrame) -> DataFrame:
    result = df.agg(
        avg("fare").alias("mean_fare"),
        stddev_pop("fare").alias("std_fare")
    ).first()

    mean_fare = result["mean_fare"]
    std_fare = result["std_fare"]

    if mean_fare is None or std_fare is None:
        return df.withColumn("is_outlier", lit(False))

    threshold = mean_fare + (2 * std_fare)

    return df.withColumn(
        "is_outlier",
        col("fare") > lit(threshold)
    )

# 36. Monthly net revenue by zone
def monthly_net_revenue_by_zone(df: DataFrame) -> DataFrame:
    if dict(df.dtypes)["trip_date"] != "date":
        df = df.withColumn(
            "trip_date",
            to_date(col("trip_date"), "yyyy-MM-dd")
        )

    return (
        df.withColumn(
            "net_revenue",
            col("fare") - col("discount")
        )
        .withColumn(
            "month",
            date_format(col("trip_date"), "yyyy-MM")
        )
        .groupBy("month", "zone")
        .agg(
            sum("net_revenue").alias("monthly_net_revenue")
        )
        .orderBy(asc("month"), asc("zone"))
    )

# 37. Weekday peak trip counts
def weekday_peak_trip_counts(df: DataFrame) -> DataFrame:
    peak_df = filter_peak_hour_trips(df)

    if dict(peak_df.dtypes)["trip_date"] != "date":
        peak_df = peak_df.withColumn(
            "trip_date",
            to_date(col("trip_date"), "yyyy-MM-dd")
        )
```

IMPLEMENTATION CODE

```
    return (
        peak_df
        .withColumn(
            "weekday",
            date_format(col("trip_date"), "E")
        )
        .groupBy("weekday")
        .agg(count("*").alias("peak_trips"))
    )

# 38. Pivot payment counts by zone
def pivot_payment_counts_by_zone(df: DataFrame) -> DataFrame:
    return (
        df.groupBy("zone")
        .pivot("payment_method")
        .count()
        .fillna(0)
    )

# 39. Calculate net revenue
def calculate_net_revenue(df: DataFrame) -> DataFrame:
    return df.withColumn(
        "net_revenue",
        (col("fare") - col("discount")).cast("double")
    )

# 40. Flag service due
def flag_service_due(
    df: DataFrame,
    due_days: int
) -> DataFrame:
    if dict(df.dtypes)["last_service_date"] != "date":
        df = df.withColumn(
            "last_service_date",
            to_date(col("last_service_date"), "yyyy-MM-dd")
        )

    days = datediff(
        current_date(),
        col("last_service_date")
    )

    return df.withColumn(
        "is_service_due",
        days > due_days
    )
```

Final Checklist

- ✓ All 40 required function names are included.
- ✓ Required aliases and tuple/list return types are followed.
- ✓ Required ranking tie-breakers are included.
- ✓ Required missing-derived-column handling is included.
- ✓ No print(), cache(), or file-writing operations are placed inside the functions.